

QUANTSIM: A HIGH-FREQUENCY TRADING SIMULATION PLATFORM

A PROJECT REPORT SUBMITTED
IN PARTIAL FULFILMENT OF THE REQUIREMENT
FOR THE AWARD OF THE DEGREE
OF

BACHELOR OF TECHNOLOGY
IN
COMPUTER SCIENCE & ENGINEERING



BY

ABHISHEK ANAND

22050440004

AMIK AFFAN

22050440017

SHAKIR AHMAD

22050440090

**DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING
CAMBRIDGE INSTITUTE OF TECHNOLOGY**

TATISILWAI, RANCHI – 835103

JHARKHAND, INDIA

[2026]

DECLARATION CERTIFICATE

This is to certify that the work presented in the project entitled “**QuantSim: A High-Frequency Trading Simulation Platform**” in partial fulfilment of the requirements for the award of the degree of Bachelor of Technology in Computer Science & Engineering at Cambridge Institute of Technology, Tatisilwai, Ranchi is an authentic work carried out under my supervision and guidance.

To the best of my knowledge, the content of this project does not form a basis for the award of any previous Degree to anyone else.

Date: _____

PROF. DEEPAK KUMAR VERMA
Dept. of Computer Science & Engineering
Cambridge Institute of Technology
Tatisilwai, Ranchi – 835103

PROF. DEEPAK KUMAR VERMA
(Head of Department)
Dept. of Computer Science & Engineering
Cambridge Institute of Technology
Tatisilwai, Ranchi – 835103

CERTIFICATE OF APPROVAL

The foregoing project entitled “QuantSim: A High-Frequency Trading Simulation Platform” is hereby approved as a creditable work on the topic and has been presented satisfactorily to warrant its acceptance as a prerequisite to the degree for which it has been submitted.

It is understood that by this approval, the undersigned does not necessarily endorse any conclusion drawn or opinion expressed therein, but approves the project for the purpose for which it is submitted.

(Internal Examiner)

(External Examiner)

HEAD OF DEPARTMENT

Dept. of Computer Science and Engineering
Cambridge Institute of Technology
Tatisilwai, Ranchi – 835103

ACKNOWLEDGEMENT

I take this opportunity to thank all who have contributed towards shaping this Final Year Project Report. I would like to express my sincere thanks to Prof. Deepak Kumar Verma (HOD, Department of Computer Science & Engineering) and to my guide Prof. Deepak Kumar Verma, whose help in the course of development of this manuscript is invaluable. As my supervisor, he/she has constantly encouraged me to remain focused on achieving my goal. I would also like to thank the technical staff members of the Department who have been kind enough to advise and help in their respective roles. I have been fortunate to have a wonderful support structure among fellow students. My sincere thanks to everyone who has provided me with kind words, new ideas, useful criticism, or their valuable time. I am truly indebted. Last but not least, I would like to dedicate this work to my family for their love and understanding.

Amik Affan

Roll No. 22050440017

ABSTRACT

High-Frequency Trading (HFT) depends on execution engines with sub-microsecond latency and faithful market-microstructure modelling, yet the mechanisms by which leading quantitative firms operate, limit order books, matching engines, queue priority, and inventory-aware market making, remain hidden inside proprietary, costly infrastructure. Compounding this, most accessible backtesting frameworks operate on aggregated Open-High-Low-Close (OHLC) candle data, ignoring the bid-ask spread, price-time priority, queue position, partial fills, transaction costs, and latency. These simplifications systematically overstate strategy performance and produce backtests that fail to generalize to live markets.

This dissertation presents QUANTSIM, an event-driven HFT simulation and backtesting platform that closes these gaps. The system is built around a low-latency C++ core exposing a full Limit Order Book (LOB) with First-In-First-Out (FIFO) price-time priority and a synchronous matching engine supporting eight order types (Limit, Market, IOC, FOK, Post-Only, Stop, Stop-Limit, Iceberg). A realistic execution layer models probabilistic maker fills, queue position, slippage, fees and rebates, borrow costs, and configurable latency, and a pre-trade risk engine adds a lock-free kill switch, rate limiter, position and notional limits, and a stale-price guard.

The platform ships five strategies, Avellaneda-Stoikov market making, Ornstein-Uhlenbeck pairs trading, TWAP execution, a multi-asset mean-reversion portfolio, and a Black-Scholes options market-maker with delta hedging, and bridges its C++ core to Python via pybind11 so that strategies and analytics are authored in Python while executing on the low-latency engine. A GPU-rendered Dear ImGui/ImPlot interface provides thirteen analytical tabs, an in-application C++ strategy compiler, and a Bookmap-style liquidity heatmap, and the same strategy binary runs unchanged in both the backtester and a C++-native live Binance paper-trading harness.

Empirical evaluation shows a matching-engine throughput above 14 million operations per second at a median latency below 100 nanoseconds, with all 27 unit tests passing. QUANTSIM thus unifies order-book-level realism, reproducibility, and interactive analysis in one free, open, and extensible platform, giving students and researchers direct access to execution mechanics previously confined to proprietary systems.

Keywords: High-Frequency Trading, Limit Order Book, Matching Engine, Market Microstructure, Avellaneda-Stoikov, Statistical Arbitrage, Event-Driven Backtesting, Low-Latency C++, pybind11, Dear ImGui.

TABLE OF CONTENTS

Declaration Certificate	i
Certificate of Approval	ii
Acknowledgement	iii
Abstract	iv
Table of Contents	v
List of Figures	viii
List of Tables	viii
List of Abbreviations	ix
CHAPTER 1 – INTRODUCTION	1
1.1 Background and Motivation	1
1.2 Problem Statement	3
1.3 Objectives of the Project	3
1.4 Scope of the Project	4
1.5 Organization of the Report	5
CHAPTER 2 – LITERATURE REVIEW	6
2.1 Related Works	6
2.1.1 Market Microstructure Theory	6
2.1.2 Optimal Market Making	6
2.1.3 Statistical Arbitrage and Pairs Trading	7
2.1.4 Optimal Execution	7
2.1.5 Options Pricing and Greeks	8
2.1.6 Low-Latency Systems Engineering	8
2.2 Comparative Analysis	8
2.3 Gap Identification	9
CHAPTER 3 – SYSTEM ANALYSIS AND DESIGN	10
3.1 System Requirements	10
3.1.1 Functional Requirements	10
3.1.2 Non-Functional Requirements	11

3.1.3	Hardware and Software Environment	12
3.2	System Architecture	12
3.3	Data Flow Diagrams	13
3.3.1	Level 0 (Context Diagram)	13
3.3.2	Level 1 (Detailed Diagram)	14
3.4	UML and ER Diagrams	14
3.4.1	Use Case Diagram	15
3.4.2	Class Diagram	15
3.4.3	Sequence Diagram	16
3.4.4	Entity-Relationship View	17
3.5	Design Patterns and Implementation Decisions	17
CHAPTER 4 – IMPLEMENTATION		19
4.1	Technologies and Tools Used	19
4.2	Module Description	20
4.2.1	Order and Fill Representation	20
4.2.2	Limit Order Book	21
4.2.3	Matching Engine	21
4.2.4	Market Data Feeds	21
4.2.5	Risk Engine	22
4.2.6	Metrics and Reporting	22
4.2.7	Python Bindings and Research Layer	22
4.2.8	Desktop Graphical Interface	22
4.2.9	In-Application C++ Strategy Compiler	24
4.2.10	Live Paper-Trading Harness	25
4.3	Algorithms and Methodology	25
4.3.1	Order Matching Algorithm	25
4.3.2	Avellaneda-Stoikov Market Making	26
4.3.3	Statistical Arbitrage	27
4.3.4	TWAP Execution and Implementation Shortfall	28
4.3.5	Black-Scholes Options Pricing	28
4.3.6	Performance Metrics	28
4.4	Code Implementation	29
4.5	Multi-Asset Portfolio and Options Strategies	32
4.5.1	Cross-Sectional Mean-Reversion Portfolio	32
4.5.2	Options Market-Making with Delta Hedging	33
4.6	Challenges Faced and Resolved	33
4.6.1	Matching Engine Correctness	33
4.6.2	Real-Time Graphical Interface	34

CHAPTER 5 – TESTING AND RESULTS	35
5.1 Testing Strategy	35
5.2 Test Cases and Results	36
5.3 Performance Analysis	37
5.4 Strategy Backtest Results	39
5.5 Exported Tear Sheet from a Logged Run	42
5.6 Live Paper-Trading Validation	44
CHAPTER 6 – CONCLUSION AND FUTURE WORK	45
6.1 Conclusion	45
6.2 Project Significance and Broader Impact	46
6.3 Contribution Notes	46
6.4 Future Enhancements	47
6.5 Closing Remarks	48
References	49
APPENDIX A – SOURCE CODE HIGHLIGHTS	51
A.1 Order Type Enumeration	51
A.2 Black-Scholes Greeks	51
A.3 Performance Metrics in Python	53
APPENDIX B – SAMPLE OUTPUTS	54
B.1 Benchmark Console Output	54
B.2 Tear-Sheet Metrics Output	54
B.3 Order Book Depth Snapshot	55
B.4 Multi-Strategy Comparison	55
B.5 Parameter Sweep Optimizer	56

LIST OF FIGURES

Figure 3.1 – Four-layer architecture of QUANTSIM	13
Figure 3.2 – Level 0 (context) data flow diagram	14
Figure 3.3 – Level 1 data flow diagram	14
Figure 3.4 – Use case diagram	15
Figure 3.5 – Class diagram of the C++ core	16
Figure 3.6 – Sequence diagram for the order lifecycle	16
Figure 3.7 – Entity-relationship view of the platform’s persisted artifacts	17
Figure 4.1 – The QUANTSIM desktop interface (Charts tab)	24
Figure 5.1 – Sample unit-test output	37
Figure 5.2 – Matching engine match latency percentiles	38
Figure 5.3 – Throughput comparison (C++ matching vs. Python backtester)	39
Figure 5.4 – Representative equity curves for three strategies	40
Figure 5.5 – Results tab of the desktop interface	41
Figure 5.6 – Profit-and-loss curve of an actual logged portfolio-strategy run	43
Figure 5.7 – Live paper-trading session on the Binance BTCUSDT feed	44
Figure B.1 – Compare tab of the desktop interface	56
Figure B.2 – Optimizer tab of the desktop interface	57

LIST OF TABLES

Table 2.1 – Comparison of trading simulation and backtesting frameworks	9
Table 3.1 – Functional requirements of QUANTSIM	11
Table 3.2 – Hardware environment	12
Table 3.3 – Software environment	12
Table 4.1 – Technology stack and justifications	20

Table 5.1 – Order book unit test cases and results	36
Table 5.2 – Matching engine unit test cases and results	36
Table 5.3 – Measured performance characteristics	38
Table 5.4 – Risk-adjusted performance of backtested strategies (synthetic data) 41	
Table 5.5 – Metrics from the exported tear sheet of a logged portfolio run (verbatim from JSON export)	43
Table 6.1 – Primary contribution areas by team member	47
Table B.1 – Sample order book depth snapshot (five levels per side)	55

LIST OF ABBREVIATIONS

Abbreviation	Full Form
ABI	Application Binary Interface
API	Application Programming Interface
AS	Avellaneda-Stoikov (market-making model)
ATM	At-The-Money
BS	Black-Scholes
CARA	Constant Absolute Risk Aversion
CLI	Command-Line Interface
CPU	Central Processing Unit
CSV	Comma-Separated Values
DFD	Data Flow Diagram
DMA	Direct Market Access
FIFO	First-In, First-Out
FOK	Fill-Or-Kill
FPS	Frames Per Second
GBM	Geometric Brownian Motion
GPU	Graphics Processing Unit
GUI	Graphical User Interface
HFT	High-Frequency Trading
IOC	Immediate-Or-Cancel
IS	Implementation Shortfall

Abbreviation	Full Form
ITCH	Nasdaq market-data binary protocol
IV	Implied Volatility
JSON	JavaScript Object Notation
LOB	Limit Order Book
MM	Market Making
OHLC	Open-High-Low-Close
OU	Ornstein-Uhlenbeck (process)
PnL	Profit and Loss
RNG	Random Number Generator
RSI	Relative Strength Index
SIMD	Single Instruction, Multiple Data
SMA	Simple Moving Average
TCA	Transaction Cost Analysis
TLS	Transport Layer Security
TWAP	Time-Weighted Average Price
VaR	Value at Risk
VWAP	Volume-Weighted Average Price
WSS	WebSocket Secure